LATER GATOR

DOCUMENTATION REPORT

PREPARED FOR

CEN3031 – Introduction to Software Engineering
FALL 2025 | University of Florida
Professor Lisha Zhou

PREPARED BY

TEAM LATER GATOR (05):
Danielle Foege
Madison Holt
Carrie Ruble
Samantha Wong

REMOTE REPOSITORY

https://github.com/smwong36/Later-Gator

RELEASE ARCHIVE

https://github.com/smwong36/Later-Gator/releases/tag/v0.1.0-pre

Cover image created with ChatGPT (OpenAI).

TABLE OF CONTENTS

INTRODUCTION

This documentation report communicates the design, architecture, and initial, prototype-level implementation of ***Later Gator***, a mobile screen-time awareness tool developed for CEN3031: Introduction to Software Engineering. Throughout the semester, our team applied core software-engineering practices, including SCRUM, version control, iterative development, testing, architectural modeling, and documentation, while building a functional prototype aligned with real user needs.

Because none of our team members had prior mobile-development experience, this project provided an opportunity to learn new technologies from the ground up. As we progressed, we encountered several significant technical roadblocks, including dependency conflicts, build failures, and incompatibilities between React Native, Room, and various Android toolchains. These challenges required us to pivot our architecture mid-semester into a native Android Studio + Kotlin + SQLite environment. Although this pivot led to substantial rework, it ultimately strengthened the project by giving us stable access to Android APIs needed for usage tracking, permissions handling, and background monitoring.

The resulting product is a prototype intended to demonstrate the feasibility of ***Later Gator's*** core functionality rather than a fully production-ready application. While not all user stories have been implemented, the prototype successfully includes core features such as authentication, usage tracking, Pomodoro study mode, app-limit popups, snooze logic, and a functioning local database. More importantly, the development process closely mirrored real-world software-engineering challenges, giving the team authentic experience in planning, collaboration, problem-solving, and adaptive project management.

The sections that follow document our project vision, implementation details, architecture, testing strategy, challenges encountered, and opportunities for future development.

SECTION 1: PROJECT DESCRIPTION

Later Gator is a screen time awareness tool designed to help users recognize when their device use shifts from intentional activity to passive scrolling. Many people report that they open an app for a quick task, only to realize much later that they have continued browsing without noticing the passage of time. This pattern can interfere with focus, academic work, sleep hygiene, and overall well-being. Our project focuses on addressing this specific moment of transition: the point when use stops being deliberate and becomes automatic.

The team identified the client need early in the semester. Students and young adults are particularly vulnerable to losing time on social media and entertainment apps. Existing solutions often provide information only after the time has already been spent, such as daily summaries or post-use alerts. Other tools feel punitive, which leads many users to disable them. There is a clear gap in tools that support real-time awareness without forcing behavior through harsh restrictions.

To respond to this challenge, ***Later Gator*** was designed with a simple concept. The app monitors per-app usage and presents an intervention at the moment a user reaches their self-defined limit. The intervention offers choices. The user can take a break, begin a focus timer, or snooze the limit if they truly need more time. The goal is not to punish; it is to interrupt the unconscious pattern in a way that encourages reflection.

The prototype includes several core features. These features include the login system, early versions of the settings screen, Pomodoro Study Mode, usage tracking, snooze logic, app limit popups, and basic reporting functions. The prototype also includes the database foundation that supports not only these features but also the features described in all previouisly-defined user stories. Other planned features remain in progress or are noted as future work in later sections of this report.

Project Vision

The project addresses the client's needs / challenge statement…

FOR students and young adults ***WHO*** struggle with excessive or unconscious screen time, ***Later Gator****, the screen time awareness tool* **is an Android app *THAT*** tracks per-app usage and offers real-time, opt-in interventions to exit distracting apps, start a focus timer, and capture quick reflections. ***UNLIKE*** phone-native settings buried in menus or "nag only" apps, ***OUR*** product lets users set limits, take decisive breaks, and build healthier habits with simple prompts and data they control.

Features & Functionality

The features of ***Later Gator*** were developed from a set of user stories created early in the semester. These user stories helped the team define the goals of the prototype, understand user needs, and break the functionality into smaller, actionable tasks. Not all features were completed within the timeline of the course, which is expected for a prototype that required both toolchain transitions and new technical learning. The following subsections outline the features that were successfully implemented and those that remain as future work.

Implemented Features

The following features are functional in the current prototype. They represent the core experience that ***Later Gator*** aims to support.

Security

Data Protection - Authentication

As a user, I want assurance that my data is secure so I can trust the app with my information.

- Google Sign-In using Firebase Authentication
- Returning users are recognized and bypass the login screen
- All data stored locally on device
- No third-party sharing
- New users can complete introductory screens before accessing the home interface

Setup & Settings

Settings Functionality

As a new user, I want an initial setup plan so I can establish baseline goals and personalize the app from the start.

- Users can see their current permissions and grant missing ones
- Users can adjust several preferences related to app tracking and time limits
- Settings update the database and reflect state changes in the UI

App Limit Popups

- When a user reaches their defined daily limit for an app, the intervention screen appears
- The popup provides three actions: exit, snooze, or continue

Overrides

Customizable Snooze Limits

As a user, I want to set the number of times I can snooze before lockout so I can avoid endless postponing.

- Snoozes are counted and stored in the database
- If snooze limits are reached, the app correctly blocks further continuation

Customizable Modes

Pomodoro Study Mode

As a user, I want a screen that has a mascot to implement proven focus intervals so I can study better.

- Users can launch a Pomodoro session from the home screen
- The timer runs for a chosen study interval followed by a break interval

Deferred Features

The team was ambitious in its early planning and proposed a wide range of features to support mindful device use. After transitioning from React Native to native Android, several high-level goals were reevaluated. The team prioritized stability, reliable usage tracking, accurate database integration, and a working Pomodoro system that could demonstrate the core concept of intentional device use. This prioritization resulted in a meaningful and functional prototype that meets the essential goals of Later Gator, even though several advanced features remain as future enhancements.

The following features may be integrated in future development cycles:

Bypass

Emergency Bypass

As a user, I want to bypass a time limit in case of an emergency without feeling like I am "cheating" the system.

Tracking & Reports

App Usage Breakdown

As a user, I want to see and analyze what apps dominate my screen time so that I know which ones to target for limits.

Daily and Weekly Tracking

As a user, I want to see a daily and weekly breakdown of my screen time so that I can understand my habits and track my progress toward healthier device use.

Notifications & Check-ins

Reflective Check-ins

As a user, I want random short check-ins ("How focused are you right now?") so I can stay mindful of my habits.

Morning/Evening Reflections

As a user, I want quick daily check-ins (e.g., top 3 things on my to-do list for today/tomorrow) so I stay intentional with my time.

Sleep Hygiene Reminders

As a user, I want reminders to wind down and stop using my phone before bed so I can improve my sleep quality.

Notification Engagement Level

As a user, I want control over how many notifications I receive so I don't feel overwhelmed or distracted.

Positive Reinforcement

Rewards & Motivation

As a user, I want rewards when I meet my goals so I feel motivated to continue improving my screen time habits.

Blocking Periods

Scheduled Blocking

As a user, I want to schedule blocking periods so I can create routines and stay focused at specific times of day.

Calendar Integration

As a user, I want the app to sync with my school calendar or to-do list so it knows when I should be focusing on work.

System Models

In exploring software architecture options for the ***Later Gator*** prototype, the team compared several design approaches, including microservices, client-server, and event-driven architectures. Because our app runs entirely on a single device and does not rely on distributed services, those models were unnecessarily complex for our goals. The chosen pattern, offers a more practical and maintainable fit, allowing us to keep components modular while simplifying development and deployment for a mobile environment.

Architectural Pattern

Later Gator uses a layered monolithic architecture that organizes responsibilities into clearly defined components. This structure helps maintain modularity while ensuring that all core functionality can run entirely on the user's device. A layered approach allows the team to separate concerns, simplify troubleshooting, and integrate new features without disrupting existing components. The architecture includes four primary layers and one crosscutting layer that provides security and external integrations.

This pattern was selected because ***Later Gator*** does not require distributed services and does not rely on backend infrastructure beyond authentication. The application is designed to function offline once the user has logged in, and all behavioral data remains on the device. This directly supports user privacy and aligns with the app's goal of reducing intrusive monitoring.

Presentation Layer:

The Presentation Layer manages the user interface and all user interactions. It includes screens for login, onboarding, home, settings, reports, and Pomodoro Study Mode. The layer presents data returned from the Business and Data layers and translates user actions into meaningful events that the Application Layer processes.

Examples include:

- Rendering usage summaries and Pomodoro history
- Displaying permission status
- Presenting app limit popups
- Handling buttons for Exit, Snooze, Start Study Session, and View Reports

The Presentation Layer communicates only with the Application Layer, which prevents UI logic from influencing deeper business rules.

Application Layer:

The Application Layer serves as the coordinator for all app behavior. It manages navigation, permissions, timers, redirect logic, and the movement of data between the UI and the Business and Logic Layer. It acts as the intermediary between user actions and the underlying business rules.

Responsibilities include:

- Managing screen transitions
- Routing requests for database reads and writes
- Launching the monitoring service
- Handling Pomodoro timers and break intervals
- Coordinating the logic behind snooze attempts and app exit actions

This layer ensures that ***Later Gator*** behaves consistently regardless of how users move through the interface.

Business and Logic Layer:

The Business and Logic Layer enforces ***Later Gator's*** rules. It contains the core decision-making algorithms that define how the app responds to usage patterns, limits, snoozes, and study sessions.

Key responsibilities:

- Determining when a daily app limit has been reached
- Deciding when a snooze is allowed or blocked
- Assessing timer outcomes for Pomodoro intervals
- Structuring data for reports
- Selecting which events should be written to the ledger tables
- Coordinating limit detection in the background service

This layer is responsible for correctness, reliability, and maintaining a predictable user experience.

Data Layer:

The Data Layer manages storage and retrieval of all information used by the application. ***Later Gator*** uses a local SQLite database file that is created when the app is first installed. This database stores user preferences, app usage sessions, snooze outcomes, Pomodoro activity, and weekly summaries.

Examples of managed tables:

- $usage\_sessions$ for tracking active foreground app sessions
- $snooze\_ledger$ for storing snooze activity
- $time\_limit\_prefs$ for daily and weekly limit values
- $pomodoro\_ledger$ for work and break interval history
- $weekly\_snapshots$ for cached report data

The Data Layer includes SQL queries, prepared statements, and utility functions written in Kotlin for insertion, updates, and retrieval. The data definition summary is provided in Appendix A.

External Services/Cross-Cutting Layer:

This layer provides shared functionality that supports security, authentication, permissions, and background execution. It is relevant across every other layer in the system.

Components include:

- **Firebase Authentication** for secure login
- **Android AccessibilityService** to detect app transitions
- **UsageStatsManager** to collect foreground app usage
- **Foreground Service** to maintain continuous monitoring
- **Android Notification Channels** for report reminders and limit popups
- **SharedPreferences** for lightweight cached values

This layer enables ***Later Gator*** to operate reliably and securely while respecting user privacy.

System Context Model

Later Gator operates primarily on the user's device and relies on only one external service, which is Firebase Authentication. The rest of the system exists entirely within the app boundary. The model below describes the major actors and their interactions with the system. The overall system boundary and external entities are shown in Figure 1.

Actors and External Entities:

- **End User:** interacts with the UI, accepts permissions, reviews reports, and responds to prompts.
- **Firebase Authentication:** verifies user identity and manages secure sign-in.
- **Android OS Services:**
  - AccessibilityService for detecting foreground apps
  - UsageStatsManager for retrieving app usage durations
  - Notification Manager for alerts
  - System clock for Pomodoro timers
- **Local SQLite Database:** stores all persistent data.

System Boundary:
The ***Later Gator*** application, which contains all layers described above, interacts with these entities. User behavior flows inward through the Presentation Layer, and data flows outward when usage events are logged or when authentication is triggered.

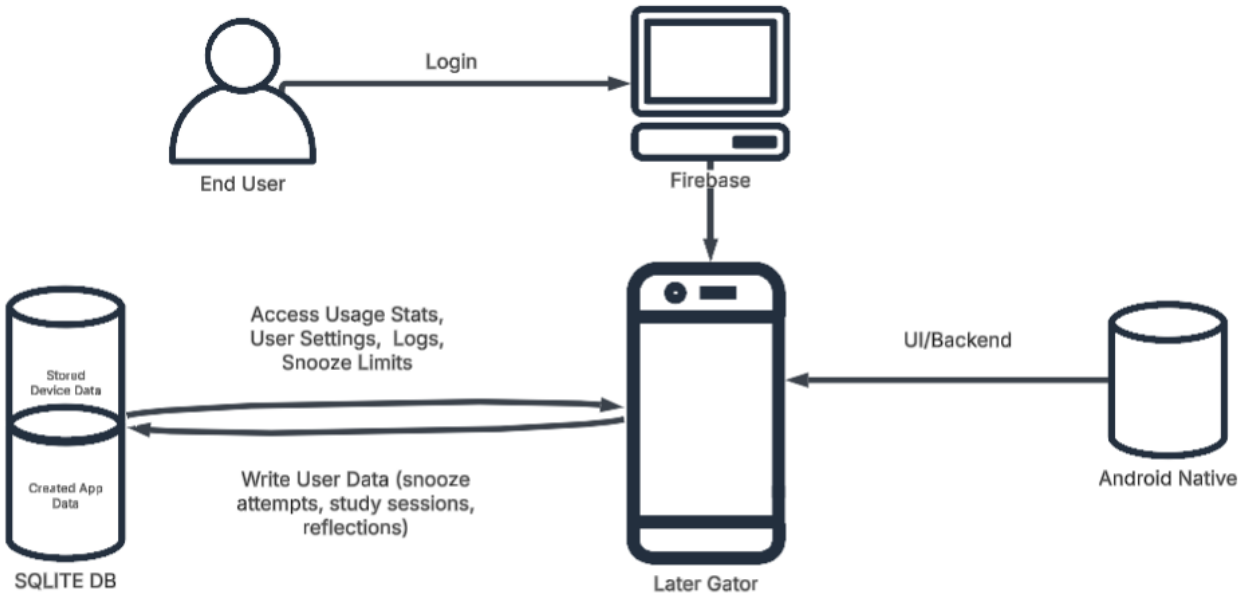

Figure 1: Updated System Context Model for Later Gator

Use Case Model

Use cases describe how the user and the system interact to achieve meaningful goals. ***Later Gator*** has several key use cases, and each reflects core functionality of the prototype.

Primary Actors:

- User
- System (automated background services)
- Firebase Authentication

Key Use Cases:

1. Log in to ***Later Gator***
2. Grant required device permissions
3. View daily or weekly usage reports
4. Begin a Pomodoro Study Session
5. Receive a limit popup for a tracked app
6. Snooze or exit a distracting app
7. Adjust settings and preferences

Team ***Later Gator*** provides the following examples, representing two features successfully implemented with our prototype:

Use Case 1: Authenticating User Upon Opening the *Later Gator* App

Primary Actor:
User

Goal in Context:
The user wants to access their personalized settings, reports, and features by successfully logging in to Later Gator.

Preconditions:

- The user has installed Later Gator on their device.
- The application is connected to Firebase Authentication.
- A valid Google account exists on the device.

Trigger:
The user selects the Later Gator app from their device's home screen or app drawer.

Scenario (Main Flow):

1. The user opens the app.
2. Later Gator checks whether the user is already authenticated.
3. If the user is not authenticated, the Google Sign-In screen appears.

4. The user selects a Google account to continue.
5. Firebase Authentication verifies the credentials.
6. If authentication succeeds, the user is directed to the Home Screen.
7. If this is the user's first time opening the app, onboarding screens appear before the Home Screen.

Exceptions and Alternate Flows:

- If no Google accounts are available on the device, the user is prompted to add one through the system settings.
- If authentication fails due to network loss or token invalidation, an error message appears and the user may retry.
- If Firebase does not recognize the app signature (such as missing SHA keys), authentication will not complete until the developer updates Firebase Console.

Postconditions:

- A successful login session is established.
- The user is granted access to settings, reports, and all available features.
- Returning users bypass the login screen during future sessions unless they manually log out.

Figure 2 illustrates the authentication process and how the user interacts with Firebase during login.

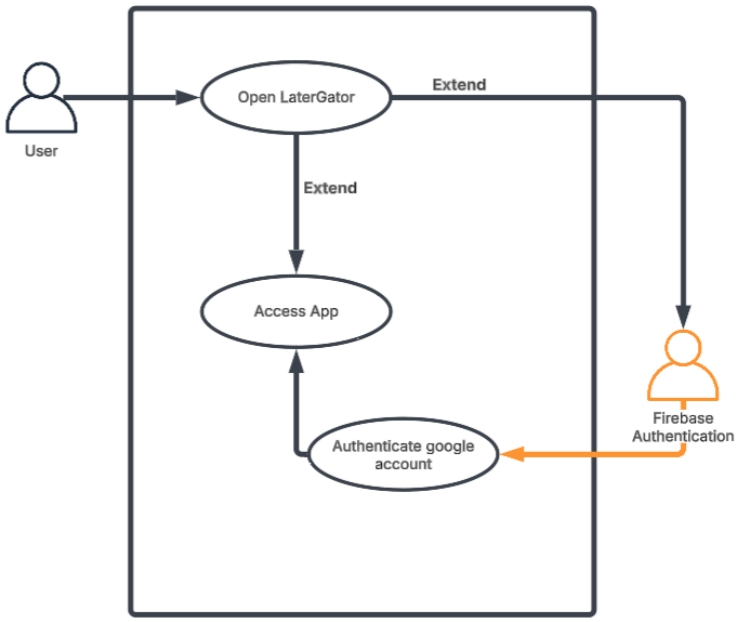

Figure 2. Use Case Diagram for User Authentication

Use Case 5: App Limit Reached and User Chooses an Action

Primary Actor:
User

Goal in Context:
The user wants to maintain healthier screen time habits by responding to a prompt when they have reached their self-defined limit for a specific app.

Preconditions:

- The user is logged in.
- The user has granted Usage Access and Accessibility permissions.
- The user has set a daily time limit for at least one app.

Trigger:
The user opens an app and reaches the daily time limit recorded in $time\_limit\_prefs$.

Scenario (Main Flow):

1. The background monitoring service detects that the user has exceeded the allotted time for the app.
2. The service triggers the limit popup.
3. The popup appears on top of the current app.
4. The user taps one of three buttons on the popup.
5. If the user selects Exit, the app is closed and the user returns to the home screen.
6. If the user selects Snooze, the Business and Logic Layer checks snooze limits.
7. If snoozes remain, the system allows the user to continue for a short interval.
8. Snooze usage is recorded in the database.
9. If snoozes are exhausted, the system exits the app and notifies the user.

Exceptions and Alternate Flows:

- If permissions are missing, the popup cannot be displayed.
- If the user selects Snooze and the limit is exhausted, the popup displays a message and forces the exit option.

Postconditions:

- A new snooze ledger entry is written, or a limit exit event is stored.
- The system updates usage sessions accordingly.

The full decision flow for this interaction is presented in Figure 3, including the exit and snooze paths.

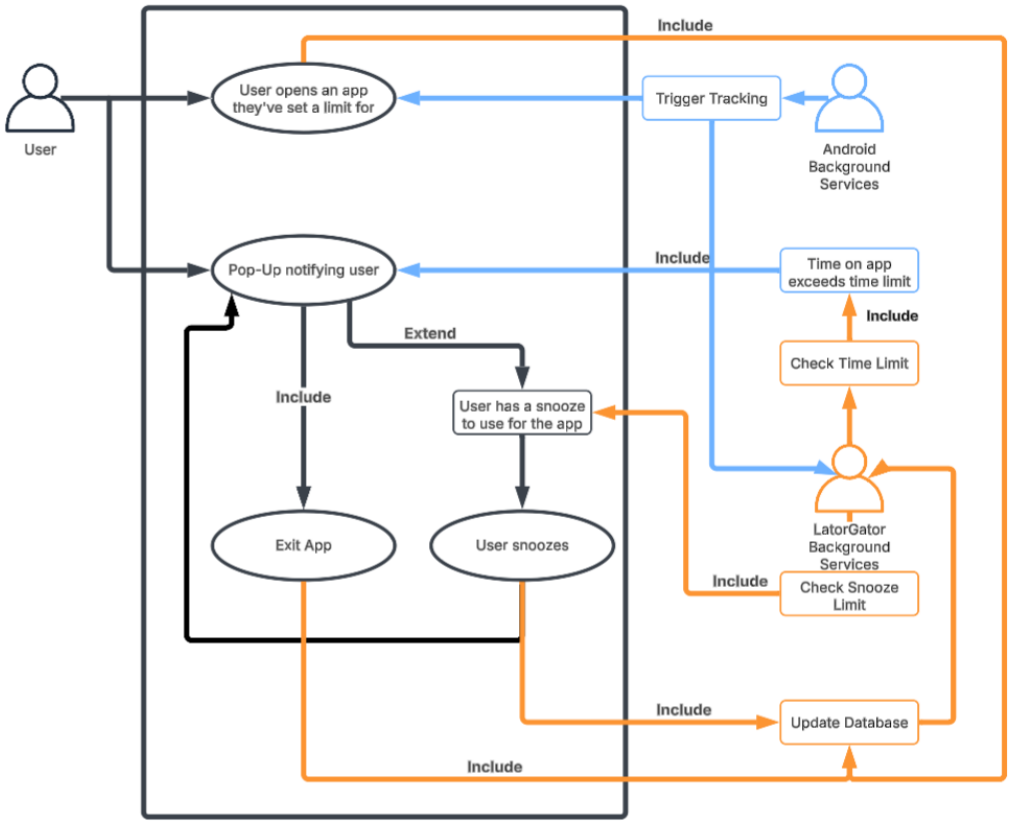

Figure 3. Use Case Diagram for App Limit Reached

SECTION 2: CODE

The ***Later Gator*** development process relied heavily on paired programming and GitHub for collaboration, version control, and organization. The team used a repository with protected branches to ensure stable development. All changes were introduced through pull requests, which encouraged team communication and prevented accidental overwrites.

Code Management

The following subsections describe how the development team organized, maintained, and collaborated on the codebase throughout the semester. Since the project required a significant architectural pivot and introduced several new tools for all team members, consistent code-management practices played a central role in keeping the project stable. These practices ensured that each contributor could work in parallel, integrate changes safely, and resolve conflicts when they appeared.

Branching Strategy

The main branch served as the stable build of the project. Team members worked in feature branches based on their assigned sprint tasks. These branches included work on login screens, Pomodoro Study Mode, navigation, database integration, settings, and the usage monitoring service. When a feature was ready for review, the developer created a pull request. Other team members reviewed the changes and tested the branch before it was approved.

This workflow helped maintain code quality and provided a natural checkpoint for identifying issues. The approach also supported pair programming and allowed more experienced teammates to help resolve conflicts. Early in the project, the team discovered several limitations with the original project structure. This experience emphasized the importance of regular merging, consistent pull habits, and clear communication about updates.

Pull Requests and Code Reviews

The team used pull requests to:

- Track changes to key files
- Document the purpose of updates
- Identify conflicts before merging
- Validate functionality through local builds
- Maintain consistency in styles and project structure

Scrum Logs show that members frequently validated each other’s environments and provided support when new features caused build failures. For example, when database

refactoring took place, team members paused their own work to verify the updated repository and troubleshoot any path or configuration issues. Some of the team's Scrum Logs have been provided in Appendix B.

Continuous Integration

The project originally included a CircleCI configuration. This configuration was intended for running automated checks; however, as the project migrated to native Android development, the CI pipeline became less critical because the team needed to validate changes manually within Android Studio and the emulator. The strategy remained useful because it supported a disciplined process for code review and protected the main branch from unstable commits.

Collaboration Practices

The team used a combination of GitHub, Jira, Slack, and weekly Scrum meetings via Zoom to coordinate tasks. Scrum Logs reflect how code management challenges influenced sprint planning. Improper environment setup or mismatched dependencies created delays, but the GitHub workflow provided a strong foundation for recovering quickly. The team learned to use issues, comments, and structured reviews to maintain progress and control scope.

Test Plan

Testing played an essential role in validating the stability of the prototype. Although the project is not yet feature complete, the team created a set of structured test cases that cover the major functionalities. These test cases focus on UI navigation, authentication, settings, Pomodoro Study Mode, snooze logic, and usage reports. The full test case table is included in the Appendix C.

Testing Methodology

The team used a combination of manual and automated testing. Manual tests were executed regularly during development to confirm that navigation and interactions behaved as expected. Automated tests in the _*tests*_ suite were created early in the project, although the migration to native Android reduced the usefulness of the original React Native test environment.

Tests were organized according to:

- Preconditions
- Actions taken during the test
- Expected results
- Postconditions

Examples include:

- Verifying Google login through Firebase
- Confirming the navigation buttons load the correct screens
- Ensuring usage reports pull correct data from the database
- Checking Pomodoro intervals for both completed and incomplete sessions
- Testing snooze limits through real popups

Testing Challenges

Several challenges influenced the testing process.

1. **Android Permissions**
   The Usage Access and Accessibility permissions are controlled by the device, and the app cannot simulate these settings. This required repeated testing with the emulator and sometimes a physical device.
2. **Background Services**
   Because monitoring relies on a foreground service, certain behaviors only appear after prolonged app usage. This increased the time required to validate expected outcomes.
3. **Refactoring and Database Integration**
   When the team migrated out of Room and into SQLite, many tests had to be repeated or rewritten. Manual testing helped validate that new database tables were correctly created, updated, and queried.
4. **Cross-Environment Consistency**
   Scrum Logs show that each teammate experienced different emulator behavior or build errors. This created delays because tests had to be repeated in multiple environments before they could be considered stable.

Testing Outcome

Although not all features were fully implemented, the existing test suite, combined with manual tests, verified that:

- Main navigation works correctly
- Authentication with Firebase is reliable
- Pomodoro Study Mode logs consistent data
- Snooze limits function as expected
- Reports pull meaningful usage data

These tests confirm the functionality of the MVP and serve as a foundation for future automated testing.

Static Code Analysis and Report

Static code analysis refers to the process of examining source code without running the application. The purpose is to detect potential vulnerabilities, structural weaknesses, and

maintainability concerns early in development. These tools often identify issues such as inconsistent naming, duplicated logic, inefficient queries, and possible security risks in data handling.

Although our team did not perform a full static code analysis during this prototype cycle, the project would benefit from conducting one in a future sprint. The database layer, in particular, would be a high priority for review. Some SQL statements are not fully uniform, and several queries could be made more consistent or modular. Our current $DatabaseHelper.kt$ file handles too many responsibilities in a single class. A static code analysis would help identify opportunities to separate this logic into smaller, purpose-specific components in order to improve readability, maintainability, and adherence to separation of concerns.

If the project were to continue, running a static analysis tool and refactoring the database layer would be one of the first technical tasks the team would complete.

SECTION 3: TECHNICAL DETAILS

The ***Later Gator*** prototype was developed using a collection of mobile development tools, platform-level system APIs, and local storage technologies. This section outlines the tools, technologies, and architectural choices that support the current version of the application.

Tools & Technologies

The tools used in this project evolved significantly as the team adapted to technical challenges. Early in the semester, the team expected to build ***Later Gator*** in React Native with a Room database. As dependency issues grew more severe, the team worked together to evaluate alternatives and ultimately decided to migrate to native Android development. This pivot required learning new tools while still making progress toward sprint goals. The transition demonstrated how well the team communicated, divided responsibilities, and supported one another through unfamiliar technical environments. The tools described in the following subsections represent the final, stable configuration that enabled the ***Later Gator*** prototype to function as intended.

Android Studio (Native Development Environment)

Later Gator was developed using Android Studio, which provided access to the Android SDK, Gradle build tools, debugging utilities, and device emulators. The native environment allowed the team to access system-level features that were not easily supported in the earlier React Native setup. These features include foreground app monitoring, UsageStats permissions, and Accessibility interactions.

Android Studio also provided resource management, UI rendering tools, XML layout editors, and built-in code inspections. These tools helped the team identify potential build issues early in development.

Kotlin

Kotlin was used as the primary programming language for all application logic. Kotlin's concise syntax, interoperability with Java libraries, and support for Android Jetpack components made it well suited for this project. Kotlin also integrates cleanly with SQLite and foreground services, both of which are central to ***Later Gator's*** functionality.

SQLite Local Database

Later Gator uses a pre-seeded SQLite database that is created on first launch. The database is stored entirely on the user's device in order to preserve user privacy. All usage sessions, snooze activity, Pomodoro logs, limit preferences, modes, and reflection entries are written to this database.

The database design includes more than twenty tables to support tracking, reporting, settings, and optional features. While the prototype does not yet use every table, the structure provides a foundation for future expansion. The

The team acknowledges that the SQL schema is not fully uniform. If development continued, one of the first technical tasks would be standardizing SQL statements and separating the $DatabaseHelper.kt$ file into smaller components that follow separation of duties.

Firebase Authentication

Firebase is used exclusively for user authentication. The team configured the project to accept Google Sign-In credentials, which provides a simple and secure method for identifying users. No user behavioral data is stored in Firebase. Instead, all usage data is stored locally on the device.

Gradle Build System

The Gradle build system manages dependencies, compiles project modules, and defines the application configuration. The team worked extensively with Gradle settings during the migration out of React Native. The final build structure includes only Kotlin-based modules and the required manifest entries for permissions, services, and activities.

Version Control with GitHub

GitHub was used for branch management, code sharing, pull requests, and version tracking. Branch protection rules ensured that only reviewed code was merged into the main branch. This tool supported collaborative development and helped the team recover quickly from build failures.

Android Emulators and Physical Devices

Testing relied on a combination of emulators and physical devices. Some permissions, such as AccessibilityService interactions and certain notification behaviors, were more reliable when tested on a physical device. Emulators were primarily used for navigation, login, and initial usage tracking.

System APIs Used

Later Gator depends on several Android system APIs to monitor app usage, display notifications, and provide real-time interventions. No third-party API keys were used.

UsageStatsManager API

This API provides access to system usage patterns. The app uses it to:

- Identify which app is currently in the foreground
- Calculate the duration of active sessions
- Produce daily and weekly usage summaries

AccessibilityService API

AccessibilityService enables ***Later Gator*** to detect real-time app transitions. The service monitors when apps move into the foreground, which allows the system to determine whether a limit has been reached.

Foreground Service API

The monitoring logic runs inside a Foreground Service so that the system keeps the process alive even when the user moves between apps. This ensures that time tracking and limit detection remain accurate.

Notification and Alert APIs

Later Gator uses:

- Android Notification Channels
- System notifications
- Overlay permissions for limit popups

This combination supports both reminders and intervention screens.

Android System Clock and Handler APIs

Timer operations for Pomodoro Study Mode rely on the system clock and scheduled callbacks. This ensures that intervals continue running even when the app is minimized.

SharedPreferences

Lightweight data, such as onboarding status and cached settings, is stored in SharedPreferences. This prevents unnecessary repeated database queries for simple checks.

SQLite and Cursor APIs

Android's built-in SQLite libraries support database creation, queries, updates, and cursor management. These APIs form the backbone of the Data Layer.

Installation Instructions

The ***Later Gator*** repository contains both the Android Studio project and a collection of course deliverables. To simplify installation and avoid requiring graders to navigate the full repository structure, a standalone ZIP file containing only the runnable Android project has been provided through the GitHub Releases page.

The ZIP file can be downloaded from the **Later Gator Prototype Pre-Release**, available here: **https://github.com/smwong36/Later-Gator/releases/tag/v0.1.0-pre.**

This ZIP includes only the files required to open and run the application in Android Studio. Course documentation, SCRUM reports, and other non-code artifacts are not part of the pre-release package.

Step 1: Download the Pre-Release ZIP

1. Navigate to the Later Gator GitHub Releases page.
2. Select **"Later Gator Prototype Pre-Release."**
3. Download the attached ZIP file.
4. Extract the contents to a location of your choice.

Step 2: Open the Project in Android Studio

1. Launch Android Studio.
2. Choose **"Open an existing project."**
3. Select the extracted folder that contains the Android Studio project files.
   This folder includes:
   - *app/*
   - *gradle/*
   - *build.gradle*
   - *settings.gradle*

Step 3: Allow Gradle to Sync

Android Studio will prompt you to sync the project.
Accept any SDK updates or Gradle tool installations that appear.

Step 4: Configure a Device or Emulator

1. Create an Android emulator (API Level 30 or higher is recommended).
2. Alternatively, connect a physical Android device with USB debugging enabled.

Step 5: Run the Application

Select ***Run*** in Android Studio. The ***Later Gator*** prototype will install automatically on the device or emulator.

Step 6: Grant Required Permissions

Later Gator requires three device-level permissions to function properly:

- Usage Access
- Accessibility Access
- Overlay Access

Users will be prompted to grant these permissions through the system settings interface on first use.

Step 7: Begin Using the Prototype

Once permissions are granted, the user can log in with any Google account and begin exploring the available features, including Pomodoro Study Mode, app limit popups, settings, and pomodoro mode.

Login and Access Credentials

Login

Later Gator uses Google Sign-In through Firebase Authentication.
To test the app during development, the team used:

- Personal Gmail accounts
- Shared test accounts created for the project

Any Gmail account is acceptable as long as the SHA fingerprints of the build are registered within the Firebase Console. The team updated the SHA keys after migrating to Android Studio, which ensured the app could authenticate correctly.

Password Handling

Password handling is not part of this application. All identity verification is managed through Firebase's secure OAuth process.

APIs

Instead of API keys, the project relies on:

- UsageStatsManager
- AccessibilityService
- Foreground Service
- Notification Manager
- SQLite
- SharedPreferences
- System Clock and Handlers
- Firebase Authentication

These APIs enable Later Gator to monitor app habits, run timers, present real-time interventions, and maintain user data securely on the device.

SECTION 4: RISK & QUALITY CONSIDERATIONS

This section provides an overview of the risks identified throughout the development of ***Later Gator*** and the software quality attributes that guided our decisions during the project. Early in the semester, the team created an initial Risk Management Plan to anticipate challenges related to technology, tools, scope, scheduling, and user privacy. As development progressed, several of these risks emerged in unexpected ways, particularly during the transition from React Native to native Android development. By revisiting the original plan and documenting what actually occurred, the team gained a clearer understanding of the constraints that shaped the prototype.

In addition to risk considerations, this section also highlights the software quality attributes that influenced our design choices. These attributes serve as benchmarks for evaluating the stability, usability, maintainability, and long-term viability of the application. Together, the risk analysis and quality attributes provide a comprehensive view of the factors that affected project outcomes and the lessons that will inform future development.

Risk Management Plan

This section revisits the original Risk Management Plan created early in the semester. At that time, our team attempted to anticipate the most likely challenges we would face. Several of those risks materialized during the project, while others emerged unexpectedly as we shifted technologies and refined our workflow. The following subsections describe the risks we originally identified, what we actually encountered, and the updated guidance we would follow if development continued in a future sprint.

1. Technology Risk

Original Expectation

We anticipated moderate difficulties related to learning new tools. Our early plan identified potential issues with dependency management, platform incompatibility, and limited experience with mobile development.

What Actually Happened

Technology risk became the most significant obstacle throughout the project. The initial React Native environment revealed incompatibilities between Android Gradle plugin versions and various dependencies. These issues created build failures that consumed a large portion of our available development time. After multiple attempts to resolve the conflicts, the team migrated to native Android development using Kotlin and SQLite. This transition resolved the stability issues, but required many components to be rebuilt from the beginning.

Updated Technology Risk Guidance

If the project is continued, future development should begin with a clear definition of required libraries, toolchain versions, and system APIs. New features should be introduced cautiously and tested immediately on both the emulator and a physical device. The team should also maintain a record of environment settings so future contributors can recreate the project setup without repeated trial and error.

2. People Risk

Original Expectation

We expected scheduling constraints and conflicting course requirements might affect team availability. Our plan emphasized communication and consistent check-ins through Scrum meetings.

What Actually Happened

The People Risks were manageable. Team members consistently attended meetings and supported one another when technical obstacles appeared. However, the time required to troubleshoot the development environment reduced the availability for some teammates to focus on feature implementation. Workloads from other courses also influenced the pace of the project during certain weeks.

Updated People Risk Guidance

To manage People Risks in future sprints, the team should adopt shorter development cycles with frequent branch merges. Early environment validation and paired setup sessions would reduce the amount of time individuals spend resolving configuration problems alone. A clear division of responsibilities would also help balance the workload.

3. Organizational Risk

Original Expectation

We assumed the project scope might shift as we learned more about mobile development. We planned to adjust features when necessary and maintain a realistic view of what could be completed within the semester.

What Actually Happened

Organizational Risk became relevant when the architecture pivoted from React Native to native Android. The migration required a re-evaluation of scope, and several planned features were deferred. The Scrum Logs show that our sprint goals changed multiple times, which influenced how we allocated effort during the second half of the project.

Updated Organizational Risk Guidance

Future work should begin with a revised project roadmap that reflects the stable Android environment. Features should be prioritized based on technical feasibility and importance to the user experience. When unexpected scope adjustments occur, the team should document the change and communicate the impact to the rest of the group before proceeding.

4. Tools Risk

Original Expectation

We expected some risk associated with unfamiliar tools, especially the Android development environment, GitHub workflows, and Firebase Authentication.

What Actually Happened

Tools Risk was closely tied to Technology Risk. GitHub played a major role in helping the team recover from unstable builds, but we also encountered several merge conflicts, incorrect project paths, and difficulties with environment setup. Android Studio eventually provided a more reliable toolset than our original approach, but there was a significant learning curve during the transition.

Updated Tools Risk Guidance

Future development should include a standardized repository structure, clear guidance on where to open the project in Android Studio, and detailed onboarding instructions for new contributors. Developers should test small changes frequently and avoid long-lived branches that become difficult to merge.

5. Requirements Risk

Original Expectation

We identified that requirements might evolve as we gained more understanding of what was technically possible. We also anticipated that some features might need to be simplified.

What Actually Happened

The requirements changed significantly once the team realized the limitations of the initial environment. Certain advanced features, such as multi-level limits, scheduled modes, and detailed visual reporting, were postponed. The team prioritized core functionality: authentication, Pomodoro sessions, app usage monitoring, limit popups, snooze logic, and basic reports.

Updated Requirements Risk Guidance

Requirements for future development should be reviewed in light of the native Android architecture. The team should document required API calls, database tables, and system interactions before implementing new features. All new requirements should be validated through small prototypes or proofs of concept to reduce the chance of encountering large setbacks.

6. Estimation Risk

Original Expectation

Our initial project plan estimated that the team could complete all major features, including advanced reporting, scheduled modes, and category tracking, within the semester. These estimates were based on our early user stories and a general assumption that mobile development would progress steadily.

What Actually Happened

The estimates proved to be optimistic. The amount of time required to resolve environment issues and dependency conflicts was significantly higher than expected. The migration from React Native to native Kotlin development reset much of the project timeline. Several sprints were spent troubleshooting build tools rather than producing new features. This resulted in reduced capacity for implementing extended functionality and placed greater emphasis on building a stable minimum viable product.

Updated Estimation Risk Guidance

If development continues, the team should generate new time estimates based on the completed MVP. Estimates should be grounded in known complexities of Android development, such as permission handling, services, background tasks, and database operations. Story points should be adjusted to reflect realistic labor requirements, and prototypes should be completed for any future feature that depends on complex system APIs before the team commits to a full implementation.

7. Ethical / Data Privacy Risk

Original Expectation

We noted that ***Later Gator*** deals with sensitive behavioral information. The early plan emphasized minimizing data collection and storing all usage information on the user's device. We expected privacy considerations but believed they could be addressed through local storage and clear explanations of how data is handled.

What Actually Happened

The privacy plan worked as intended. No user data is transmitted to external services, and the prototype stores all usage logs, snoozes, Pomodoro activity, and limit preferences exclusively in the local SQLite database. This approach aligns with the purpose of the application and protects the user from unnecessary data exposure. However, privacy-related complexity increased once system-level permissions were introduced. Usage Access and Accessibility Access allow the app to detect foreground applications, and these permissions can appear sensitive to users. Although ***Later Gator*** does not transmit this information externally, the team recognized the importance of transparency.

Updated Ethical / Data Privacy Risk Guidance

Future work should include an explicit Privacy Statement within the app. This statement should explain which system permissions are required, how the data is used, and why it never leaves the device. If new features are added that require cloud synchronization or multi-device access, the team will need to incorporate secure storage practices, encryption at rest, and explicit user consent. The team should also perform periodic reviews of platform-level privacy changes introduced by the Android operating system to ensure that ***Later Gator*** remains compliant and respectful of user expectations.

Summary Risk Reflection

In reviewing our original Risk Management Plan, the team gained a deeper understanding of how software projects evolve when unexpected challenges appear. Technology and Tools Risks became the most significant barriers, while People and Organizational Risks were manageable through communication and flexibility. If development continues in a future sprint, the updated risk strategies in this section will help stabilize the project and guide contributors toward a more predictable and efficient workflow.

Software Quality Attributes

Software quality attributes provide a framework for evaluating how well a system supports reliability, usability, maintainability, and other characteristics that contribute to an effective design. These attributes are especially important in projects that evolve over time or require future enhancements. Although ***Later Gator*** is a prototype, many of these attributes influenced our decisions throughout development and shaped the direction of the system architecture.

The following summarizes the most relevant software quality attributes for this project and describe how each one relates to the current implementation.

Maintainability

Maintainability refers to how easily a system can be modified, extended, or understood. ***Later Gator's*** layered architecture supports maintainability by separating UI components, business logic, and data storage. The migration to native Android development increased clarity in the codebase, since Kotlin provided consistent patterns and reduced ambiguity. Some areas, such as the database helper utilities and SQL statements, would benefit from further refactoring. If the project continues, dividing the database logic into smaller components will improve readability and support future enhancements.

Reliability

Reliability focuses on the system's ability to perform its intended functions under expected conditions. The prototype demonstrates reliable behavior for core features such as login, Pomodoro timers, limit popups, and usage tracking. The team encountered early reliability challenges when toolchain conflicts disrupted the build environment. Once the project stabilized in native Android, the system became more predictable. Reliability will improve further as background monitoring logic becomes more robust and additional tests are added.

Usability

Usability refers to how intuitive and efficient the interface is for end users. The current prototype includes a simple navigation flow through the home screen, settings, Pomodoro mode, and reports. The design focuses on clarity and straightforward actions, which is important for users who may already feel overwhelmed by device use. Future work should include improved visual reports and guidance that explains the purpose of each permission, since users may be unfamiliar with Usage Access and Accessibility Access.

Efficiency

Efficiency relates to how well the system uses device resources such as memory, battery, and processing time. Native Android development improved efficiency by allowing direct access to Android system APIs without the overhead of additional bridging layers. The foreground service that handles app monitoring is designed to remain active without unnecessary processing. Additional profiling would help identify opportunities to reduce power usage, especially during long monitoring sessions.

Integrity

Integrity concerns the accuracy and protection of data. ***Later Gator*** stores all usage and behavioral information locally within a secure SQLite database. This decision protects user privacy and reduces the risk of external exposure. The system prevents unauthorized modification by relying on controlled database operations. A future enhancement would

include validation layers to ensure that all inputs, including limit preferences and notes, remain consistent with expected formats.

Portability

Portability refers to the ability of the system to operate on different devices or environments. The prototype currently supports Android devices and has been tested on both emulators and physical hardware. The use of native Android tools and Kotlin improves compatibility across a range of device versions. Expanding portability to iOS would require a separate development effort. Since all usage tracking features depend on Android-specific APIs, portability across platforms is limited by design.

Testability

Testability focuses on how easily the system can be tested. The modular structure of ***Later Gator*** improves testability by providing clear entry points for interaction. Manual testing was effective for verifying logic in Pomodoro sessions, login flows, and snooze limits. Automated testing will be more achievable once the database logic and monitoring services are fully refactored into smaller components. Improved testability will also support long-term reliability and maintainability.

Summary Quality Attributes

The ***Later Gator*** prototype demonstrates several positive quality attributes, particularly maintainability, reliability of core features, and protection of user privacy. Other attributes, such as efficiency and testability, will continue to improve with future development. By grounding the design in proven software quality principles, the team created a solid technical foundation that can support continued expansion and refinement.

CONCLUSION

The ***Later Gator*** prototype represents the combined effort of a team working through new technologies, evolving requirements, and authentic software engineering challenges. The original vision of helping users become more aware of their screen time remained central throughout the project, even as the technical foundation shifted and the scope adjusted. By the end of the semester, the team delivered a working application that demonstrates core features such as authentication, Pomodoro Study Mode, app usage monitoring, limit popups, and snooze logic. These features reflect the essential purpose of the system and provide a functional starting point for future development.

The project also provided a meaningful opportunity to apply the principles taught in CEN3031. The team practiced SCRUM, created system models, used version control effectively, and learned to navigate complex technical decisions. The shift from React Native to native Android development required flexibility and collaboration, and it highlighted the importance of communication when working through shared obstacles. This experience mirrored professional development environments where unexpected constraints often influence architectural decisions.

Although the prototype is not feature complete, the foundation is stable and extensible. The layered architecture, database structure, and monitoring framework allow for continued progress, and the Risk Management Plan offers guidance for improving future sprints. The Software Quality Attributes discussed in this report will help direct further refinements related to maintainability, usability, testability, and long-term reliability.

Overall, ***Later Gator*** serves both as a demonstration of our progress in learning software engineering practices and as a prototype with genuine potential for future enhancement. The work completed this semester created a solid technical and conceptual base that can support additional features, improved user experience, and a more comprehensive tool for promoting intentional technology use.

REFERENCES

Google. (2024). *AccessibilityService*. Android Developers. https://developer.android.com/reference/android/accessibilityservice/AccessibilityService

Google. (2024). *UsageStatsManager*. Android Developers. https://developer.android.com/reference/android/app/usage/UsageStatsManager

Google. (2024). *Foreground services overview*. Android Developers. https://developer.android.com/develop/background-work/services/fgs

Google. (2024). *Notifications overview*. Android Developers. https://developer.android.com/develop/ui/views/notifications

Rubin, K. S. (2013). *Essential Scrum: A practical guide to the most popular agile process*. Addison-Wesley.

Sommerville, I. (2020). *Engineering Software Products*. Pearson.

Trub, L., & Barbot, B. (2022). The paradox of screen time: The relationship between digital media use and psychological well-being. *Psychology of Popular Media, 11*(3), 362–372. DOI: 10.1016/j.pmedr.2018.10.003

Ward, A. F., Duke, K., Gneezy, A., & Bos, M. W. (2017). Brain drain: The mere presence of one's smartphone reduces available cognitive capacity. *Journal of the Association for Consumer Research, 2*(2), 140–154. DOI:10.1086/691462

APPENDIX

A. DATA DEFINITION SUMMARY

(Types: INTEGER = numbers/epoch ms, TEXT = strings, JSON = small JSON blob, BOOLEAN = 0/1 in SQLite.)
(Dates use 'YYYY-MM-DD'; times use local "HH:MM"; timestamps use UTC epoch ms.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
profile	id INTEGER PK, tz TEXT, weekly_goal_minutes INTEGER, privacy_level TEXT, created_at_ms INTEGER, updated_at_ms INTEGER	Single user/device defaults (timezone, privacy, goal).	Read by reports/UI.	Keep until user clears data/uninstalls.
apps	id INTEGER PK, package_name TEXT UNIQUE, label TEXT, category TEXT, is_tracked BOOLEAN, created_at_ms INTEGER, updated_at_ms INTEGER	Catalog of tracked/limited apps.	FK target for many: usage_sessions, snooze_*, mode_settings (via JSON lists), sleep_nudges_ledger, pomodoro_ledger, emergency_allowed_apps, weekly_snapshots, etc.	Keep rows while app remains tracked; remove on user request.
time_limit_prefs	id INTEGER PK, scope TEXT, app_id INTEGER NULL, daily_limit_minutes_current INTEGER, weekly_limit_minutes_current INTEGER, daily_limit_minutes_original INTEGER, weekly_limit_minutes_original INTEGER, active INTEGER, created_at_ms INTEGER, updated_at_ms INTEGER, notes TEXT	Stores global and per-app daily/weekly usage limits, including current and original values for reporting.	FK → apps.id; used by time_limit_hits_ledger, Foreground Service, and reports.	Keep active rows until user edits or deletes limits; archived originals remain for reports.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
time_limit_hits_ledger	id INTEGER PK, at_ms INTEGER, scope TEXT, app_id INTEGER NULL, period TEXT, limit_minutes INTEGER, used_minutes INTEGER, action_taken TEXT, meta_json TEXT	Immutable record of when a usage limit was hit, used for interrupts and reports.	FK → apps.id; referenced by event logs and reports.	Keep until user clears data; optional purge of hits older than 12–18 months.
usage_sessions	id INTEGER PK, app_id INTEGER, started_at_ms INTEGER, ended_at_ms INTEGER, duration_ms INTEGER, source TEXT, blocked_flag BOOLEAN, notes TEXT	Foreground sessions; basis for daily/weekly totals.	FK→apps.id; summarized into weekly_snapshots; correlated by reports.	Keep until user clears data; optional housekeeping: purge raw sessions older than 12–18 months (snapshots preserve trends).
events	id INTEGER PK, at_ms INTEGER, kind TEXT, app_id INTEGER NULL, meta_json JSON	App "journal" (mode toggles, notif sent/suppressed, snooze used, etc.)	May reference apps; links to other ledgers via IDs in meta_json.	Keep until user clears data; optional: purge >12 months.
weekly_snapshots	id INTEGER PK, week_start_date TEXT, app_id INTEGER NULL, total_minutes INTEGER, sessions_count INTEGER, interventions_count INTEGER, avg_session_minutes REAL, created_at_ms INTEGER	Cached weekly rollups for fast charts.	FK→apps.id (nullable for "all apps"). Built from usage_sessions/ledgers.	Safe to purge/rebuild anytime; recommend keeping last 12–24 months.
mode_settings	id INTEGER PK, mode_name TEXT, enabled BOOLEAN, priority INTEGER, schedule_kind TEXT, start_time_local TEXT, end_time_local TEXT, days_mask TEXT, allow_strategy TEXT,	Defines Focus/Vacation/ etc. behavior and schedules.	Used by runtime policy engine; transitions logged in events.	Keep until user edits/deletes the mode.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
	allowed_apps_json JSON, blocked_apps_json JSON, block_notifications BOOLEAN, respect_system_dnd BOOLEAN, created_at_ms INTEGER, updated_at_ms INTEGER, notes TEXT			
snooze_prefs	id INTEGER PK, scope TEXT, app_id INTEGER NULL, max_per_day_current INTEGER, max_per_week_current INTEGER, max_per_day_original INTEGER, max_per_week_original INTEGER, created_at_ms INTEGER, updated_at_ms INTEGER, notes TEXT	User-configured snooze caps (keep originals for reports).	Optional FK→apps.id when scope='per_app'; used with snooze_ledger.	Keep current row; update in place (preserve originals).
snooze_ledger	id INTEGER PK, used_at_ms INTEGER, app_id INTEGER NULL, reason TEXT, linked_event_id INTEGER NULL, notes TEXT	One row per snooze used (immutable history).	FK→apps.id (nullable); linked_event_id→events.id.	Keep until user clears data; optional: purge >12–18 months after snapshotting.
notification_prefs	id INTEGER PK, max_per_day INTEGER, max_per_week INTEGER, quiet_start_local TEXT, quiet_end_local TEXT, reminders_enabled BOOLEAN, channel_caps_json JSON NULL, created_at_ms INTEGER, updated_at_ms INTEGER	Caps + quiet hours for app notifications (by type).	Enforced by notif pipeline; activity logged in events.	Keep until user changes prefs.
emergency_prefs	id INTEGER PK, enabled BOOLEAN, activation_note TEXT, activated_at_ms INTEGER NULL, deactivated_at_ms INTEGER NULL, priority INTEGER, created_at_ms INTEGER, updated_at_ms INTEGER	Master toggle/priority for Emergency Bypass.	Used by runtime policy; state changes logged in events.	Keep until user disables/removes feature.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
emergency_allowed_apps	id INTEGER PK, app_id INTEGER, notes TEXT	List of apps always allowed during Emergency.	FK→apps.id; consulted by policy engine.	Keep until user edits the list.
pomodoro_prefs	id INTEGER PK, work_minutes INTEGER, short_break_minutes INTEGER, long_break_minutes INTEGER, intervals_per_long_break INTEGER, auto_start_next BOOLEAN, daily_goal_intervals INTEGER, sound_enabled BOOLEAN, sound_key TEXT, sound_volume INTEGER, mascot_theme_key TEXT, created_at_ms INTEGER	User's Pomodoro timing + UI/audio choices.	Used when creating pomodoro_ledger rows; celebratory events in events.	Keep until user changes prefs.
pomodoro_ledger	id INTEGER PK, started_at_ms INTEGER, ended_at_ms INTEGER, kind TEXT, planned_minutes INTEGER, actual_minutes INTEGER, outcome TEXT, interruption_reason TEXT NULL, app_id INTEGER NULL, notes TEXT, created_at_ms INTEGER	Immutable record of each work/break interval.	Optional FK→apps.id; contributes to reports/streaks/points.	Keep until user clears data; optional: purge >12–18 months after snapshotting.
reflection_prefs	id INTEGER PK, am_enabled BOOLEAN, pm_enabled BOOLEAN, am_time_local TEXT NULL, pm_time_local TEXT NULL, created_at_ms INTEGER	Opt-in and scheduling for AM/PM reflections.	Triggers creation of reflection_sessions and goals.	Keep until user changes prefs.
reflection_sessions	id INTEGER PK, kind TEXT, at_ms INTEGER, for_date TEXT, discipline_1_5 INTEGER, motivation_1_5 INTEGER, notes TEXT	Each completed AM/PM reflection (with Likert scores).	Joins to goals via for_date; summarized into reports.	Keep until user clears data.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
goals	id INTEGER PK, for_date TEXT, text TEXT, created_via TEXT, status TEXT, created_at_ms INTEGER, completed_at_ms INTEGER NULL	Daily goals set/reviewed via reflections.	Referenced by checkin_ledger.goal_id; used in reports.	Keep until user clears data; optional: archive goals >12 months.
checkin_prefs	id INTEGER PK, enabled BOOLEAN, daily_max INTEGER, window_start_local TEXT NULL, window_end_local TEXT NULL, created_at_ms INTEGER	Randomized check-in configuration.	Drives checkin_ledger; notif events in events.	Keep until user changes prefs.
checkin_ledger	id INTEGER PK, prompted_at_ms INTEGER, goal_id INTEGER NULL, question_text TEXT, response TEXT, responded_at_ms INTEGER NULL, linked_snooze_id INTEGER NULL	Every prompt and user response.	Optional FK→goals.id; optional FK→snooze_ledger.id.	Keep until user clears data; optional: purge >12–18 months.
sleep_prefs	id INTEGER PK, enabled BOOLEAN, bedtime_local TEXT, winddown_minutes INTEGER, nudge_after_bedtime BOOLEAN, created_at_ms INTEGER	Bedtime/wind-down reminder settings.	Generates sleep_nudges_ledger; notif events in events.	Keep until user changes prefs.
sleep_nudges_ledger	id INTEGER PK, at_ms INTEGER, kind TEXT, app_id INTEGER NULL, action_taken TEXT NULL	Record of wind-down/bedtime/after-bedtime nudges.	Optional FK→apps.id; correlate with usage_sessions.	Keep until user clears data; optional: purge >12–18 months.
points_ledger *(Optional)*	id INTEGER PK, at_ms INTEGER, delta INTEGER, reason TEXT, meta_json JSON	Additive points history (rewards).	Used by UI to total points; may reference weeks/apps in meta_json.	Keep until user clears data.
streaks *(Optional)*	id INTEGER PK, type TEXT, current_len INTEGER, longest_len INTEGER, updated_at_ms INTEGER	Current/longest streak counters.	Updated from pomodoro_ledger, goals, etc.	Keep until user clears data.

Table Name	Column Names & Data Types	Purpose / Description	Relations (FK / Used by)	Retention / Deletion Rule
badges *(Optional)*	id INTEGER PK, slug TEXT, name TEXT, description TEXT, asset_key TEXT, criteria_json JSON	Badge catalog (maps to bundled assets).	Referenced by earned_badges.	Keep with app; edit only in updates.
earned_badges *(Optional)*	id INTEGER PK, badge_id INTEGER, earned_at_ms INTEGER, evidence_json JSON	Immutable log of badges earned.	FK→badges.id.	Keep until user clears data.
baseline_usage *(Optional)*	id INTEGER PK, metric TEXT, week_start_date TEXT, app_id INTEGER NULL, value_minutes INTEGER	Frozen baseline for "then vs now" comparisons.	FK→apps.id (nullable); used by reports.	Keep until user resets baseline.
security_prefs *(Optional)*	id INTEGER PK, app_lock_enabled BOOLEAN, lock_method TEXT, last_unlock_at_ms INTEGER NULL, failed_attempts INTEGER	Local app-lock (biometric/PIN) preferences.	Used only by security UI/flow; events can log unlocks.	Keep until user disables lock/clears data.

B. Weekly Scrum Logs

PROJECT NAME: Later Gator	**Later Gator**
WEEK OF: 10/19 - 10 /25	
TIME and WHERE: Zoom 7pm	

Team Member	Discussion Questions	Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Danielle Foege	What did I do since the last time we met that helped the development team meet the sprint goal?	Researched how Later Gator will access and store the data it needs.	I downloaded Android Studio and React Native.	Reviewed/prepared the Product Backlog section of the presentation and filmed.
	What will I do today to help the development team meet the sprint goal?	Start planning out aspects of the upcoming presentation like what information should be covered	Worked to setup PATH variables and run a test project to make sure everything is installed correctly.	Review our component breakdown and look into how Firebase works and other libraries that will be included.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Coordinating to create a cohesive presentation. Running into questions regarding how Later Gator will be set up and how to integrate the tools/frameworks/technologies that it will use.	Running into issues with setup. Failed connection links. General confusion because this is all new to me.	Complications in knowing what libraries to include in the Later Gator project files so that we have access to all that we need.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Madison Holt	What did I do since the last time we met that helped the development team meet the sprint goal?	Work on Presentation Slides, Check in with Canvas on Needs for later assignments	Edit and correct Presentation Slides, Set up coding environment check in with other members about needs and schedules for the next week	Review Presentation Slides and get ready for filming.
	What will I do today to help the development team meet the sprint goal?	Review my slides and add graphics	Set up meetings and setting up proper coding environment	Work with Carrie on DB, Finish Sprint 1 and rework things for Sprint 2
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Life and Personal Problems moving meeting dates	Adding too much may bring us out of scope for time. Conflicts in team schedule for next week	Technical Difficulties and unknown future conflicts
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum

Sam Wong	What did I do since the last time we met that helped the development team meet the sprint goal?	Worked on creating a script and slides for user story section of powerpoint	Ironed out script for presentation recorded and edited slides	Make sure all tools are installed. Set up environmental variables and build
	What will I do today to help the development team meet the sprint goal?	Review my slides and edit them so that my script runs more smoothly for out recording	Read through documentation in preperation to download tools for project	Brain storm insights for sprint retrospective, consider issues we may have encoutntered how our team worked through them. Pull new database commits and do a test to see if it works in my environmnet. Start coming up with test cases.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Potienally writing scripts that are too long for the presentation and running out of time. Running into personal problems that will change planned meeting dates	I may get confused or lost when setting up PATH in environmental variables	If I run into trouble pulling the new database commit. Also if I get confused using new tools
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Carrie Ruble	What did I do since the last time we met that helped the development team meet the sprint goal?	Discovered the PATH to where I have the project stored locally is too long for react-native's liking. I corrected that and then had to reconnect all tools: React-Native, Metro, and where Android Studio can find the SDK directory. I verified all was correct by changing the name of the app to "Later Gater" in the native Android file: `strings.xml`.	Had a minor heart attack, thinking that we may have forgotten to account for sending updates (future roll-outs), when we were planning. I researched Firebase Cloud Management and confirmed we will not need to change any plans for our MVP (prototype), but I did get some information we can add if needed, to our final assignment regarding "future plans."	I participated in the recording for our Design and Plan Presentation assignment, uploaded the recording to YouTube, and made necessary trims/cuts to have it submission-ready. I shared it with the team.
	What will I do today to help the development team meet the sprint goal?	Work on the slide deck for our presentation.	Finish cleaning up the slide deck and finish adding my own scripts to my slides.	I will dig deeper into what files/dependencies/API linkers need to be added to ensure our app can save data to the user's device.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Canvas was offline today. If the AWS outage continues throughout the week, that could leave assignments in other classes pliing up for the team.	I have had some unexpected personal matters come up. To focus on the presentation and other assignments due this weekend, I may ask to move SP1-10 to the next sprint as my top priority.	I think I may have underestimated the first attempts at linking all parts of the project up in the remote repo. (i.e., which files are ok to push and which shouldn't)

PROJECT NAME: Later Gator	**Later Gator**
WEEK OF: 10/26 - 11 /01	
TIME and WHERE: Zoom 7pm	

Team Member	Discussion Questions	Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Danielle Foege	What did I do since the last time we met that helped the development team meet the sprint goal?	Worked on how to setup/install Firebase to an Android Studio project that is using React /native.	Figured out the Google-sign in issue, but now having build dependency issues so working on that.	Pulled from our main branch and made sure the additions to the project ran on my branch. I created a styles.ts sheet for our project for standardized UI styles.
	What will I do today to help the development team meet the sprint goal?	Review the database for LaterGator. Continue trouble-shooting the google-sign-in	We need to come up with standardized front end styles, so I'll think about that.	Added Google-sign in feature with Firebase to the project. Added a basic navigation layout for a starting point and committed the styles sheet and these other changes.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	I'm running into issues with adding th eGoogle-sign in plugin, so that is affecting progress on the Sign-in process.	Running into issues with these new tehnologies/tools takes a while to troubleshoot because it's so new and the files for the project are complex.	I made a mistake with my commits to git that cost me time, so overall my biggest impediment right now is getting my tasks complete without running into major unexpected blocks that take a while to figure out.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Madison Holt	What did I do since the last time we met that helped the development team meet the sprint goal?	Submitted Weekly Scrum Report Sunday, Reviewed Test Case Assignment Requirements	Reviewed Test Cases, Closed out Sprint 1 and Started Sprint 2	Wrote some Test Cases, validated my branch in Github, Monitored and reviewed other main pull requests
	What will I do today to help the development team meet the sprint goal?	Review the Database Structure Carrie posted, Write Test Cases for Home and Reports, Create test gmail uf.latergator@gmail.com for test data cases, pull newestproject files from github	Rebuild Development Environment as setup was not done properly, Write test cases, monitor Sprint 2 development and update Jira accordingly	Come together with team on design and UI standards, write Test Cases, Implement Test Suite, push necessoary changes to github
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Have to study for upcoming exam and balance workload	Exam Today, Having all files in OneDrive caused my dev environment to be messed up, need to clean this up before moving forward	Assignments in other classes and other technical difficulties
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Sam Wong	What did I do since the last time we met that helped the development team meet the sprint goal?	I reviewed the material carrie posted about all the tables in our database to be implemented and the dtructure of our project.	Reviewed instructions for writing test cases	Moniter branch updates and reviewed how to add firebase to project

	What will I do today to help the development team meet the sprint goal?	I will pull Carrie's commit of the database and test out how the emulator works. I will also downloads apps. I will also work on test cases	Work on creating test cases, make sure environment and tools are downloaded and new commits are merged locally	Created test cases, corked with team to divide work and come up with test cases, installed firebase into project
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	I my job get's too hectic I may not have enough time to be as throughouh as I like	Working overtime at my job, having my file folder place too far away from the storage of the android studio build, we need to make sure where I downloaded everything	Getting confused working with new technologies and unable to understand how things work together
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Carrie Ruble	What did I do since the last time we met that helped the development team meet the sprint goal?	Created a Data Dictionary for the app database, created a SQL file for all tables, started translating to Kotlin	I was finally able to get a hybrid solution for SQL->Kotlin	Helped Team Members set up environments correctly, helped Danielle navigate issues in linking Firebase and handled github merge errors, validated pull requests from other team members
	What will I do today to help the development team meet the sprint goal?	I will finish translating the SQL into Kotlin, push the project to the repo	Work on the test case suite	Implement Test Case Suite, Assist Danielle with any further issues in implementation, Develop Test Cases with Team
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	getting the android/app package to build correctly...	learning how all the components of our app work together	Github/CircleCI has been flagging things as unable to merge, need to resolve these issues and manage pull requests of other developers while implementing the test suite

PROJECT NAME: Later Gator			**Later Gator**	
WEEK OF: 11/2 - 11/8				
TIME and WHERE: Zoom 7pm				
Team Member	Discussion Questions	Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Danielle Foege	What did I do since the last time we met that helped the development team meet the sprint goal?	Worked to develop test cases in our __tests__ file within our project. This allows us to test our features as we build them.	Looking at how the databse will connect to features in the app, specifically the settings.	Worked on connecting the frontend Settings to the databse to be stored.
	What will I do today to help the development team meet the sprint goal?	Look at the navigation setup between out different pages and how it makes sense for them to connect.	Worked to create a frontend UI for the Settings page.	Am trying to make the necessary changes to make it so that the connection between Room, Kotlin and our gradle files are correcct.

	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	I have a busy week coming up, so carving out time to work on features.	Trouble integrating frontend into backend could cause delays	Running into a issue with the build failing because the project isn't recognizing kapt/the versions of kotlin and room.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Madison Holt	What did I do since the last time we met that helped the development team meet the sprint goal?	Developed Test cases and debugged some technical difficulties with the team. Got a working main branch project with Firebase	did light research on Reactive Native Elements and set up some new elements in my personal branch.	Issues have arisen with current Database Setup. Needing to flush main and reestablish DB connections and access
	What will I do today to help the development team meet the sprint goal?	Pull main and research React Native Elements to better/more easily build UI Elements	Play around with Ui Elements in my own branch developing home screen further.	Organize Priorities on Jira, We may need to rethink some of our priorities and features to keep in this sprint as things are taking more time than expected for us. Setting up dev meetings and Reviewing Pull Requests
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	techincal difficulties and protential bugs in main gradle setup	Exam today	Family issues may pull me away from this project as well as other exams over the weekend
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Sam Wong	What did I do since the last time we met that helped the development team meet the sprint goal?	I pulled the new commit of main that allows for the emulator to be used.	Researched how to build UI screens in react native. Exerimented with builidng and tried to create a simple screen for pomodoro.	Worked on setting up the pomodor study mode screen. We ran into issues using different tools
	What will I do today to help the development team meet the sprint goal?	Experiment with react native elements to better understand how tools effect the UI.	Work on building the pomodoro screen layout and setup timer and buttons	Pull new database commit from git and adjusting to incorperate front end screens
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Running into incopatable elements when building the emulator	Buisness trip meetings may run longer than my normal workday, so I may not have enough time to work on building the UIs	Not understand how the new git push works and having issues building and merging.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Carrie Ruble	What did I do since the last time we met that helped the development team meet the sprint goal?	Implemented Test Suite and helped Debug Firebase Connectivity and my own Technical Issues	Found issues in current setup in Room Database. Rethinking the current strategy and researching possible solutions.	Tested some possible solutions for our database issues. Worked with Danielle on how the issues are appearing and what needs to be changed to solve them

	What will I do today to help the development team meet the sprint goal?	Begin Developing some features and researching plugins for React for easier UI development	Research and Implement a new database structure as the current implementation is causing errors.	Reimplement our Database structure. Everything may need to be rewritten/ Room DB may need to be removed as current structure is causing build failures.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Competiting Priorities with other classes as well as personal responsibilities	Not knowing how to better implement Room database in Kotlin and what else may be needed to fix these issues	Not knowing how to better implement Room database in Kotlin and what else may be needed to fix these issues. Not wanting to fully rework the entire System

PROJECT NAME: Later Gator			**Later Gator**	
WEEK OF: 11/9 - 11/15				
TIME and WHERE: Zoom 7pm				
Team Member	Discussion Questions	Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Danielle Foege	What did I do since the last time we met that helped the development team meet the sprint goal?	After trying to resolve the version issues between Room/KSP/KAPT/KOTLIN, it we finally realized that there were no compatible versions.	Carrie pushed the new project with the new database, so I have been working to get that set up locally. I also realized that the SSH1 key needed updated for Firebase with this new project, so I did that so that the login feature is functional again.	Added test cases to test the settings functionality. Got the app tracking, usage permission granting, and username updating working.
	What will I do today to help the development team meet the sprint goal?	Carrie and I talked and I will be available should she need help converting the project database out of Room.	I will work to add the app tracking and set time limit functionality in the settings page.	Meet with the team for our sprint retrospective and plan for our Sprint 1 presentation
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	This was a big setback that caused us create basically a whole new version of our project, so time will be tight moving forward	This new project and using a database will have a learning curve.	Testing edge cases can be tricky and for some features I need to clarify expected behavior.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Madison Holt	What did I do since the last time we met that helped the development team meet the sprint goal?	validated pull requests and submit scrum meeting log	Environment Setup and Build verified	Completed Retrospective, created new backlog items in Jira.

	What will I do today to help the development team meet the sprint goal?	resetup development environment after Refactor. We will be only using Android Studio from now on. Verify Successful Build and run some tests and demos with the team	Meet with team to complete Sprint 2 Retrospective	Validate Test Cases and pull requests submitted by Danielle, Contribute to Sprint 2 Presentation Slides, confirm build and prepare other deliverables for this week's assignments
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Personal Issues, Technical Difficulties	Personal Issues	Many deadlines this weekend, hopefully we can manage to complete all tasks this weekend
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Sam Wong	What did I do since the last time we met that helped the development team meet the sprint goal?	Updated to a newer build of our environment. Ran into problems while due to tool incompatiability between room and react native	Ran into problems with firebase and getting passed the login in screen. Discussed problems with teammates.	Prepare slides and notes for presentation. Brainstorm ideas for sprint retrospective and issues to discuss
	What will I do today to help the development team meet the sprint goal?	Work on resolving build and incompatabilty issues. Discuss with team on problems they have experienced and find solutions.	Work on understanding the firebase tool. Discuss how to apply solution and new updated to my local computer	Worked as a team on sprint retrospective and action plans for next sprint. Discussed how to split up presentation and when to record
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Confusion while working with tools and not having enough time to figure out a solution	Confusion on how to work in new environment and build errors that may occur	Lack of time due to personal deadlines and commitments.
Team Member		Monday Daily Scrum	Wednesday Daily Scrum	Friday Daily Scrum
Carrie Ruble	What did I do since the last time we met that helped the development team meet the sprint goal?	Completely reconfigured tools for project. Re: compatability issues between Room/KSP/KAPT/KOTLIN. I converted our .sql file (the INSERT TABLES database file) to a .db file using DB Browser for SQLite, and attempted conversion of firebase, google log-in, and home screen files to new configuration.	Added fingerprint to Firebase Console (per Danielle's experience) to get AndroidStudio and emulator to recognize me with a new google-services.json file. Reworked the login->NavigationManager/HomeScreen flow. Did not have a chance to push before Danielle's forward motion on her sprint tasks!	Participated in Sprint Retrospective, drafted deliverable for same, share with team for input. Shared slide deck with team. Got the app to register with AccessibilityServices to draw over other apps/run a background monitoring service w/ buttons on SettingScreen for user permission.

	What will I do today to help the development team meet the sprint goal?	I will ensure I push a working project with Room removed. This allows us to bypass the need to use Room Entities/DAOs. I will verify that the team's custom DB gets installed on the emulator device upon app install.	I will set up the Slide Deck for upcoming recording session. I will also create the Sprint Retrospective Document based on tonight's meeting. I will share both of these with the team as soon as possible.	Tracked-apps monitoring is working, writing to DB is causing app to crash, will continue to work on solution. Update my slides with images and notes on deck for presentation.
	Do I see any impediment that prevents me or the development team from meeting the sprint goal?	Based on what we just learned about version compatability, we should assume no tools will work well together.	We should also assume that the Android APIs may lead to compatibility issues with our reconfigured project. We will not meet the sprint goal, however, we previously built in a buffer for time and can move items to the next sprint. I do not believe our overall project MVP will be affected.	Need to ensure writing to DB is happening, focusing on 'securing' SQL might impede functionality - will need to balance security with 'functional' prototype for current timeline.

C. TEST CASES

PROJECT NAME	Later Gator (Group # 05)	LATER GATOR
PREPARED BY	Danielle Foege, Madison Holt, Carrie Ruble, Samantha Wong	
DATE OF CREATION	27-Oct-25	
DATE OF REVIEW	1-Nov-25	

ITEM NUMBER	TEST CASE ID	TEST CASE DESCRIPTION	PRECONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POSTCONDITION
1	2.19.1	Verify Gmail Login	Need a valid Gmail account	Click "Login With GOOGLE" button. Select a Google account.	gmail addresses: foegedanielle@gmail.com; uf.latergator@gmail.com uflatergator1, *non working email*	User is taken to the home page of Later Gator	Authentication established
2	2.19.2	Verify Logout	User is logged in	Click "Logout" button	none	User is taken back to login screen, session terminated	User authentication cleared
3	2.14.1	Verify Pomodoro Launch Button	Home Screen with "Pomodoro Study" button	from home screen click "Pomodoro Study" button and correctly launch pomodoro Module	home screen, pomodoro study button, Pomodoro Module	User is taken to Pomodoro Study Mode Start Screen	Pomodoro Module Loaded
4	2.14.2	Verify Reports Launch Button	Home Screen with "Reports" button	Click "Reports" Button and correctly launch Reports Module	home screen, "Reports" button, Reports Screen	User is taken to Reports Screen	Reports Screen Loaded
5	2.14.3	Verify Settings Launch Button	Home Screen with "Settings" Button	Click "Settings" button and correctly launch Settings Module	home screen, "Settings" button, Settings Screen	User is taken to Settings Screen	Settings Screen Loaded

6	2.14.4	Write Goal	Screen with Goals text box, submit button, goals db,	Write message, click submit, write to db, be able to recall??	"Hello World", "Hello World/n", "", "1234567890!@#$%^&*()<>?:"{}[]", *string 1000 characters long*,	User can input various text into box and text will be written to Goals db properly	Successful db write and textbox is refreshed.
7	2.14.5	Goals List	1-3 "goals" written in DB	DB entries are pulled from DB and properly formatted on Home page	"Drink Water", "Reduce Screen Time/n", Homework Assignment 1.1 @ 4pm"	DB entries from last 24 hrs are displayed on corresponding UI Element on Home Screen	Goals Displayed and Home screen Updated
8	2.14.6	Display Homescreen Gator	Home Screen	Display image/gif on home screen	Lator Gator.png, Lator Gator.gif	UI element properly displays Image/Gif	elements meant to loop properly loop and UI Elements Displayed
9	2.25.1	Call Daily Report from UsageStats	Reports Screen, UsageStats connectivity	Call UsageStats Days data	Instagram 1 hr, Candy crush 30 min, Youtube 2 hr	test data returned	
10	2.25.2	Call Weekly Report from UsageStats	Reports Screen	Call UsageStats Weeks data	D1: Instagram 2 hrs, Candy crush 1 hr, Youtube 3hrs D2: Candy crush 4 hrs, Youtube 2hrs D3: Instagram 1 hr, candy crush 30 min, Tiktok 4 hrs D4: Tiktok 6 hrs D5: 0 hrs	test data returned	
11	2.25.3	Call Pomodoro Study Session Data	Reports Screen, Pomodoro DB	Call Pomodoro DB	Pomodoro 30min , pomodoro 1 h, pomodoro incomplete, no pomodoro session	test data returned	

12	2.17.1	Display Daily Report	Reports Screen, UsageStats connectivity, Pomodoro DB	Call UsageStats Days data, Call Pomodoro Study Data, Display list in most used order, display bar graph broken by app usage	Instagram 1 hr, Candy crush 30 min, Youtube 2 hr, pomodoro 30 min	Display list in most used order, and bar graph properly colored by and broken by app usage	Report Displayed
13	2.17.2	Display Weekly Report	Reports Screen, UsageStats connectivity, Pomodoro DB	Call UsageStats Weeks data, Call Pomodoro Study Data, Display list in most used order, display bar graph broken by date, app usage	D1: Instagram 2 hrs, Candy crush 1 hr, Youtube 3hrs D2: Candy crush 4 hrs, Youtube 2hrs, pomodoro 1 hr D3: Instagram 1 hr, candy crush 30 min, Tiktok 4 hrs D4: Tiktok 6 hrs D5: 0 hrs D6: failed pomodoro D7: 1hr pomodoro, 1 failed pomodoro, 2 hrs Instagram, 37 min Candy Crush	Display list in most used order, and bar graph properly colored by and broken by date, app usage, pomodoro modes?	Report Displayed
14	2.23.1	Snooze Pop-up Exit App	Pop Permissions, Popup UI	App limit reached, display popup, press exit app	Instagram Limit 1 hr, Instagram usage 55 min; Candy Crush limit 30 min, Candy Crush usage 25 min	Popup Displays and then App is exited on button press	App Exited to Phone Home Screen
15	2.23.2	Snooze Pop-up Snooze Limit	Pop Permissions, Popup UI, Snooze DB	App limit reached, display popup, press snooze app, call Snooze limits reached from DB, check limit not reached and snooze	Instagram Limit 1 hr, Instagram usage 55 min; Candy Crush limit 30 min, Candy Crush usage 25 min; Snooze Limits = 3; Used Snoozes = 0;	Popup Displays and then App is resumed on button press, Snooze DB entry changed	App Resumed, Used Snoozes++
16	2.23.3	Snooze Pop-up Snooze Limit Reached	Pop Permissions, Popup UI, Snooze DB	App limit reached, display popup, press snooze app, call Snooze limits reached from DB, check limit reached and noptify user then exit app	Instagram Limit 1 hr, Instagram usage 55 min; Candy Crush limit 30 min, Candy Crush usage 25 min	Popup Displays and then App is exited on button press	App Exited to Phone Home Screen

17	2.18.1	Verify User can change notification frequency in settings (daily)	Notifications enabled for daily and weekly reports	Change settings to turn off daily reports, then confirm no notifications were sent	Old setting: daily report checkin turned on. New setting: daily report checkin notitfication turned off	User does not receive daily notifications after setting changed	Updated preferences saved to backend
18	2.20.1	Verify Onboarding Flow	User just logged in for the first time	Launch app after login; complete onboarding screens by swiping/clicking next	none	User completes onboarding and is directed to Home Screen	Onboarding completion status updated in DB
19	2.18.2	Verify weekly report shows	Notifications enabled for weekly reports	Make sure weekly report notification are on	Weekly report notifcation should be recieved every Saturday (end of week) at 12 pm	A push notification should appear on phone screen with reminder to view weekly report and that it has been generated	Backend setting still send weekly messages
20	2.24.1	Pomodoro Study Session	Pomodoro Module, Pomodoro DB	Select pomodoro mode from home screen, input interval, start session, let timer run, pomodoro study 25 min, break 5 min, end session, check session logged	Pomodoro interval: 25 min; Break: 5 min	Timer runs correctly; session logged in user reports	Pomodoro session recorded in DB, Return to Homescreen
21	2.24.2	Pomodoro Study Session Incomplete	Pomodoro Module, Pomodoro DB	Select pomodoro mode from home screen, input interval, start session, let timer run, end session early, check session logged	Pomodoro interval: 25 min; Break: 5 min - Time elaspsed < 30 min	Timer runs correctly; session logged in user reports	Pomodoro session recorded in DB, Return to Homescreen

22	2.24.3	Pomodoro Study Session Paused	Pomodoro Module, Pomodoro DB	Select pomodoro mode from home screen, input interval, start session, let timer run, pause session, resume, end session, check session logged	Pomodoro interval: 25 min; Break: 5 min	Timer runs correctly; session logged in user reports	Pomodoro session recorded in DB, Return to Homescreen

Test Case ID corresponds to Sprint.Item.issue